Mise en Place for Agentic Coding: Deliberate Preparation as Context Engineering Methodology

Andrew Zigler
andrew.zigler@linearb.io
LinearB
Los Angeles, USA

Abstract

The rapid adoption of AI coding agents has produced a dominant workflow pattern — often called "vibe coding" — that prioritizes speed of implementation over deliberate preparation. We argue that this approach creates a systematic alignment problem: agents that lack sufficient context produce code requiring extensive debugging and refactoring, consuming substantial development time. Drawing on the culinary concept of *mise en place* (everything in its place), we propose a three-phase preparation methodology for agentic coding: (1) contextual grounding, where domain expertise and tacit knowledge are externalized into structured documents; (2) collaborative specification, where human-agent dialogue produces detailed design artifacts; and (3) task decomposition, where specifications are converted into structured, dependency-aware task records. We report on the application of this methodology during a competitive hackathon, where two hours of preparation enabled a rapid parallel implementation of a full-stack educational platform by concurrent AI agents. We introduce the concept of *context fluency* as an emerging developer skill — the ability to create rich, structured context that agents can act on — and connect it to established frameworks in backward design and tacit knowledge externalization. We conclude with a research agenda for empirically validating preparation-phase methodologies in AI-assisted software development.

CCS Concepts

• **Software and its engineering** → **Software creation and management**; **Collaboration in software development**; • **Human-centered computing** → *Human computer interaction (HCI).*

Keywords

agentic coding, mise en place, context engineering, vibe coding, context fluency

1 Introduction

AI coding agents have become remarkably capable at code generation. Controlled studies of GitHub Copilot report productivity gains of 21–55% across programming tasks [12], and the broader ecosystem of autonomous agents can now scaffold entire applications from natural-language descriptions. Yet the dominant workflow pattern for leveraging these capabilities—what Karpathy [9] termed "vibe coding"—prioritizes speed of implementation over deliberate preparation. The developer describes an intent, the agent produces code, and misalignments are resolved through iterative correction. This creates a systematic alignment problem: agents operating without sufficient context produce code that requires extensive rework. Veracode reports that 45% of AI-generated code contains security flaws [16], and practitioners consistently observe that debugging misaligned agent output can consume substantial development time [8, 18]. The bottleneck in agentic coding is not code generation but alignment between developer intent and agent output—that is, the degree to which agent-produced artifacts exhibit specification compliance (planned features built as specified), architectural fidelity (design decisions honored across the codebase), and low rework ratio (corrective commits as a proportion of total commits).

We argue that this alignment problem is fundamentally a *preparation* problem. Drawing on the culinary concept of *mise en place*—a French term meaning "everything in its place"—we propose a preparation-first methodology for agentic coding. In professional kitchens, thorough preparation before service enables fluid execution: every ingredient is measured, every tool is positioned, so that once cooking begins, the chef's hands never pause to search. The same principle applies to agent orchestration. When domain expertise, design intent, and task boundaries are externalized into structured artifacts *before* agents begin writing code, the resulting implementation is more aligned, more coherent, and less costly to verify. Our methodology formalizes this preparation phase into three distinct steps: contextual grounding, where tacit knowledge and domain expertise are captured in machine-legible documents; collaborative specification, where human-agent dialogue produces detailed design artifacts; and task decomposition, where specifications are converted into structured, dependency-aware work records.

This paper makes four contributions:

(1) A three-phase mise en place methodology for agentic coding, grounded in backward design [17] and tacit knowledge externalization [14].
(2) A case study applying this methodology in a competitive hackathon, with quantitative data on preparation artifacts and implementation outcomes.
(3) The concept of *context fluency* as an emerging developer skill—the ability to create rich, structured context that AI agents can act on.
(4) A research agenda with five open questions for empirical validation of preparation-phase methodologies.

2 Background and Related Work

Early empirical studies of AI-assisted coding reported substantial productivity gains: Peng et al. found that developers using GitHub Copilot completed tasks 21–55% faster [12]. However, speed has not translated cleanly into quality. Veracode's 2025 analysis found that 45% of AI-generated code contains security flaws [16], while Mozannar et al. showed that developers' mental models frequently diverge from actual AI behavior, leading to inappropriate acceptance of incorrect suggestions [10]. Vasconcelos et al. demonstrated

that trust in AI code generation depends on context-aware explanations that current tools rarely provide [15]. Together, these findings point to a productivity paradox: AI agents accelerate output while simultaneously increasing the surface area for rework and defects. The bottleneck in agentic coding, we argue, is not code generation but *alignment*—ensuring that what agents build matches what practitioners intend.

Several preparation-oriented methodologies have emerged in response. Horthy's Research-Plan-Implement (RPI) methodology [7] most directly anticipates our work, requiring practitioners to complete discrete research and planning phases before any code is generated. Mise en place differs from RPI by formalizing tacit knowledge externalization in Phase 1 and insisting on backward-design sequencing where all three phases complete before implementation begins. GitHub's Spec Kit [6] formalizes specification-driven development, where a structured specification artifact governs agent behavior throughout implementation. More broadly, the discourse has shifted from prompt engineering to *context engineering*: designing the complete informational environment surrounding an LLM [2, 5]. Alomar et al. extend this by proposing *promptware engineering* [1], while Dakhel et al.'s taxonomy identifies eleven distinct human-AI interaction types [3]. These contributions recognize that what happens *before* agent execution shapes outcomes, but none offers a unified, theoretically grounded preparation methodology.

A parallel stream of practitioner innovation addresses preparation through tooling. Karpathy coined "vibe coding" [9] for a minimal-preparation workflow he scoped to throwaway projects, yet the term came to characterize the dominant mode of AI-assisted development. At the opposite end, Huntley's Ralph Loop [8] provides fine-grained iterative task delegation, and Yegge's Beads framework [18] addresses agent memory through structured task records, extended by Emanuel's Rust reimplementation [4]. These tools solve specific problems—iteration, memory, orchestration—but assume that appropriate context already exists.

Our methodology draws on two theoretical frameworks. Wiggins and McTighe's *backward design* [17] holds that effective instruction begins with desired outcomes, then works backward through assessment criteria—a deliberate inversion of the "content-first" approach. We apply this principle: defining agent outputs and success criteria before designing context. Polanyi's tacit knowledge—"we can know more than we can tell" [13, 14]—frames preparation as a knowledge externalization problem. Agents cannot access knowledge that has not been made explicit; preparation is the act of making it so. Nonaka and Takeuchi's SECI model [11] formalizes this tacit-to-explicit conversion as a repeatable organizational process, providing additional theoretical grounding for Phase 1's knowledge externalization step.

These four streams—empirical AI-assisted development research, emerging preparation methodologies, practitioner tooling, and educational theory—converge on the importance of deliberate preparation but remain fragmented. No existing work unifies them into a formalized methodology with theoretical grounding and empirical demonstration. Our mise en place framework addresses this gap.

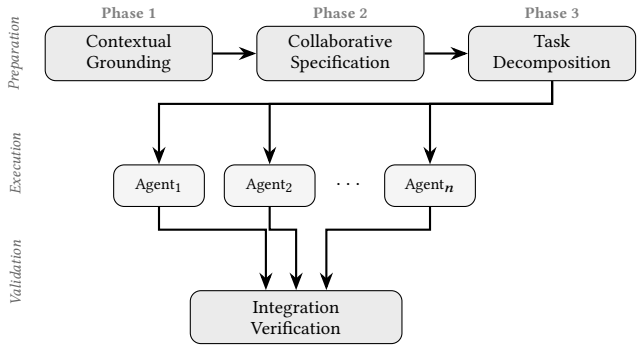

Figure 1: The mise en place methodology. Three sequential preparation phases produce structured artifacts consumed by parallel agents. Task decomposition distributes independent work units to n concurrent agents, whose outputs converge in integration verification.

3 The Mise en Place Methodology

We propose *mise en place* as a structured preparation methodology for agentic coding. The term, borrowed from professional culinary practice, describes the discipline of arranging every ingredient and tool before cooking begins. In a professional kitchen, the speed of service is a direct function of the thoroughness of preparation: chefs who invest time in mise en place execute complex dishes fluidly, while those who skip preparation spend service time searching, measuring, and recovering from avoidable errors. We observe an analogous dynamic in AI-assisted software development: when practitioners invest in deliberate preparation—externalizing domain knowledge, producing detailed specifications, and decomposing work into structured task records—the resulting agent execution is faster, more aligned with intent, and requires less corrective iteration.

The methodology formalizes preparation into three sequential phases, each producing defined artifacts that feed into the next. Inspired by the backward design principle [17], the phases are completed before any implementation begins. Figure 1 illustrates the three phases and their relationship to agent execution.

3.1 Phase 1: Contextual Grounding

The first phase externalizes domain expertise and tacit knowledge into structured documents that agents can consume. Its purpose is to capture context that agents cannot generate independently—what Polanyi [14] characterized as tacit knowledge, the understanding that practitioners possess but struggle to articulate. In agentic coding, this includes institutional context, domain-specific constraints, competitive landscape awareness, design philosophy, and the practitioner's accumulated judgment about what constitutes quality in the target domain.

The artifacts produced are briefing documents: markdown files encoding domain knowledge, competitive analysis, architectural preferences, and design philosophy. These documents serve a dual purpose. First, they force the practitioner to articulate knowledge that would otherwise remain implicit. Second, they create a persistent context layer that agents reference throughout implementation,

reducing the need for repeated clarification. Following backward design [17], contextual grounding begins with outcomes (the "why") rather than technical features (the "what"), establishing alignment criteria that propagate through subsequent phases.

3.2 Phase 2: Collaborative Specification

The second phase produces detailed design artifacts through structured human-agent dialogue. The practitioner describes intent; the agent proposes implementation details; the practitioner accepts, rejects, or modifies the proposal. This iterative refinement continues until both parties converge on a specification detailed enough for implementation without further clarification.

The key mechanism is the encoding of value judgments as specification constraints. What to *exclude* is as important as what to include. Scope management—the deliberate decision to constrain features, simplify interactions, or defer capabilities—represents a class of judgment that agents cannot make independently. The output is a comprehensive design document that specifies screens, interactions, data flows, and quality standards, capturing not just the *what* but the *why* behind each decision. This distinction separates collaborative specification from conventional requirements gathering: the specification encodes rationale, enabling agents to make aligned micro-decisions during implementation without escalating to the practitioner.

3.3 Phase 3: Task Decomposition

The final preparatory phase converts the specification into structured, dependency-aware task records. We employ Beads [4, 18]—lightweight JSON records backed by Git, carrying priorities, dependencies, and acceptance criteria—though the principle generalizes to any structured task system.

Fine-grained decomposition enables parallel agent execution without coordination overhead. When each task is independently executable—with clear boundaries, defined inputs and outputs, and explicit dependencies—multiple agents can work simultaneously. The coordination burden shifts from runtime (agents negotiating) to preparation time (the practitioner defining task boundaries), where human judgment about system architecture is most valuable. The output of this phase is a dependency graph where each node represents an independently executable unit of work. The graph structure enables the practitioner to identify the critical path, allocate agents to parallelizable tasks, and monitor progress through structured completion signals. It also provides a natural mechanism for prioritization: core features are executed first, while stretch goals are explicitly deferred, preventing scope creep during implementation.

Following parallel agent execution, an integration verification stage validates outputs against the specification. The practitioner reviews agent-produced code, files defects as new task records, and dispatches agents for targeted corrections.

The three phases of mise en place draw individually on established work—tacit knowledge externalization [13], the RPI methodology [7], and iterative task delegation [8]. The methodology's contribution is their integration into a sequential, phase-gated process with defined artifacts and inspiration from backward design: the insistence that all three phases complete before implementation begins.

Table 1: Hackathon timeline and artifact summary.

Phase	Duration	Key Artifacts
Contextual grounding	~2 hr	10 docs (9,386 words)
Collab. specification	(concurrent)	Product specification
Task decomposition	(concurrent)	64 beads w/ dependencies
Agent implementation	184 min	43 beads closed (med. 5.9 min)
Deployment & polish	52 min	9 commits, 6 bugs resolved

4 Case Study: Competitive Hackathon

We report on the application of mise en place during a competitive hackathon to illustrate the methodology's behavior under realistic constraints. The setting imposed fixed time limits, external evaluation, and a requirement to produce working software—conditions that favor rapid iteration and penalize unproductive preparation.

4.1 Setting

The hackathon took place in January 2026, organized around a publisher's editorial archive and an AI-powered content API. Approximately 12 teams competed in a five-hour window to build working prototypes. A panel of industry judges evaluated final presentations. The practitioner-researcher had professional backgrounds in both education and software engineering, a combination that informed the product concept and design decisions throughout the event.

4.2 Preparation Phase

While most teams began coding within the first fifteen minutes, the practitioner-researcher spent approximately two hours in deliberate preparation, applying the three phases described in Section 3.

Contextual grounding produced 10 planning documents totaling 9,386 words, including API exploration notes, competitive analysis, and—critically—an extended dictation on pedagogical design philosophy drawn from the practitioner's teaching experience, encoding tacit knowledge about how effective learning environments create conditions for student thinking [14].

Collaborative specification occurred concurrently, as the practitioner refined the product concept through dialogue with an AI agent. The resulting specification described a platform where teachers curate bounded research environments from The Atlantic's archive and students conduct research using a Socratic AI tutor. Key value judgments encoded in the specification included constraining the product to a single assignment type (prioritizing depth over breadth for a three-minute demo), insisting on real API data rather than mocks, and scoping the analytics dashboard to show the student's research *journey* rather than only the final submission.

Task decomposition converted the specification into 64 structured task records with priorities and dependencies. The planning-to-code ratio—9,386 words of planning against 8,496 lines of source code—was 1.10:1. Table 1 summarizes the timeline.

4.3 Implementation and Results

Four parallel subagents were deployed, each responsible for a distinct feature area: classroom creation, student research workspace, day-one/day-thirty demo toggle, and Socratic AI tutor. The agents

drew on the contextual grounding documents and specification produced during preparation.

Bead closure timestamps provide a granular implementation timeline. During a 184-minute active window (19:41–22:45 UTC), agents closed 43 beads with a median closure time of 5.9 minutes. Closures occurred in five parallel clusters—the largest comprising 10 beads closed within 21 seconds of each other, spanning five features and five supporting tasks across UI, state management, and API layers. Of 43 closures, 42 occurred before the first git commit at 22:24 UTC, indicating that the bead system—not version control—served as the primary progress tracker. The subsequent deployment phase produced 9 commits over 52 minutes. The final codebase comprised 43 TypeScript/TSX files totaling 8,496 lines deployed to production.

Of 64 hackathon-day beads, 43 were closed (67.2%). The 21 remaining open beads were stretch features explicitly deprioritized during decomposition. All 6 bugs filed during the event were resolved.

The resulting prototype was a full-stack educational platform with multiple user roles, AI-assisted research tools, and a Socratic tutor component.

Three observations are relevant to the methodology's evaluation. First, the architecture required no structural refactoring during the deployment phase. The bugs that emerged were integration and styling issues—API content truncation, data flow between components, favicon configuration—not architectural misalignment. This suggests that the preparation phase successfully transferred the practitioner's architectural intent to the agents. Second, the preparation-to-execution time ratio was approximately 5.7:1 (two hours of preparation against approximately 21 minutes of active implementation per agent, across four parallel agents), indicating a deliberate investment directionally opposite to the "start coding immediately" pattern common in hackathon settings. Bug-type beads were resolved with a median closure time of 1.2 minutes (mean 1.5 minutes), compared to 9.7 minutes for implementation tasks, suggesting that defects introduced during parallel execution were quickly identified and corrected. Third, near-zero architectural rework is consistent with the hypothesis that rich upfront context reduces agent misalignment, though we cannot establish causation from a single case study.

4.4 Limitations

This case study is illustrative rather than definitive. It involves a single practitioner, a single project, and no control group. We cannot determine whether the outcome would have differed with less preparation, a different practitioner, or a different AI model. The hackathon setting, while providing authentic time pressure, is not representative of sustained multi-month development. We present these results as suggestive evidence that the methodology merits further investigation, not as proof of its general efficacy. Additionally, the practitioner's dual professional background in education and software engineering likely contributed to the quality of contextual grounding, particularly the pedagogical domain encoding that informed the Socratic tutor. Whether mise en place yields equivalent results for practitioners without specialized domain expertise remains an open question, directly motivating RQ3. Nevertheless, the methodology's emphasis on deliberate preparation stands in marked contrast to the iteration-first patterns documented in practitioner literature [9]—a contrast that itself motivates formal comparative study.

5 Context Fluency: An Emerging Skill

Our case study and the broader practitioner literature suggest that effective agentic coding requires a skill set distinct from both traditional programming and prompt engineering. We propose the term *context fluency* to describe this capability: the ability to create rich, structured context that AI agents can act on effectively. Context fluency encompasses capturing domain expertise, value judgments, and design intent in machine-legible form—not as one-shot prompts, but as persistent informational environments within which agents operate. Prompt engineering optimizes individual instructions; context fluency is *upstream* of prompting, concerned with the entire informational architecture surrounding agent execution [2, 5].

Where the methodology (Section 3) describes a process of sequential phases producing defined artifacts, context fluency describes the practitioner skill that makes the process effective. The distinction parallels that between a recipe and culinary judgment: the recipe structures the work, but the outcome depends on the cook's ability to execute it.

We identify four components of context fluency:

Decomposition: Breaking problems into discrete, parallelizable tasks with clear boundaries, enabling concurrent agent execution.

Specification: Describing not just *what* to build but *why*, so agents can make aligned micro-decisions without human intervention.

Constraint definition: Knowing what to exclude, simplify, or defer—scope management as a first-class concern.

Domain encoding: Externalizing tacit knowledge [14] that agents cannot generate on their own—directly instantiating what Nonaka and Takeuchi [11] term *externalization*, converting tacit knowledge into explicit, communicable form. Twenty minutes of dictated pedagogical intuitions produced context that substantially reduced iteration on the domain-specific tutor component.

These components map to established frameworks. Backward design [17] contributes outcome-driven preparation. Polanyi's tacit knowledge [13] frames the externalization challenge: practitioners "know more than they can tell," and context fluency is the discipline of telling it anyway. We also observe a connection to pedagogical scaffolding: context fluency resembles lesson planning, where the teacher structures an environment so the learner (or agent) can do meaningful work independently. Both disciplines are fundamentally about enabling others to act within a defined informational context.

If context fluency is a distinct and identifiable skill, the implications extend beyond individual practice. For developer education, it suggests that curricula should cultivate specification, decomposition, and domain-encoding abilities alongside traditional programming competencies. For hiring, it implies that practitioners with strong domain knowledge and pedagogical instincts—the ability to

structure information for consumption by others—may be disproportionately effective in agentic workflows [3]. For tool design, it reframes preparation aids such as structured task systems [4, 18], specification frameworks [6], and context engineering platforms [2] as essential infrastructure rather than optional enhancements.

6 Research Agenda and Conclusion

6.1 Research Agenda

Mise en place as described here rests on a single case study and conceptual argument. Empirical validation across practitioners, domains, and agent models is essential. We propose five research questions:

RQ1: *Does deliberate preparation reduce agent misalignment compared to iterative development, and at what cost-benefit threshold?* Controlled experiments comparing preparation-first and iteration-first workflows could establish whether the upfront investment yields net savings.

RQ2: *What constitutes "sufficient" context for agentic work—can we define stopping criteria for the preparation phase?* A saturation-style analysis measuring alignment quality as a function of context volume would help practitioners know when to stop preparing.

RQ3: *How does context fluency vary across practitioners, domains, and agent models?* Comparative studies could reveal whether context fluency is trainable or a function of prior domain knowledge [13].

RQ4: *Can mise en place scale beyond prototyping to sustained, multi-month development?* Longitudinal case studies would test the methodology's applicability at scale [3].

RQ5: *What is the relationship between preparation time and implementation quality across project complexities?* A multi-project dataset correlating preparation artifacts with implementation metrics could yield quantitative models of the preparation-quality relationship [1].

These questions span experimental, observational, and longitudinal designs. RQ1 is the most immediately tractable, addressable with paired tasks and a modest participant pool.

6.2 Conclusion

Mise en place, as presented here, is a starting framework rather than a finished methodology—a formalization of practices that experienced practitioners already converge on independently [7, 8, 18]. The methodology formalizes what Nonaka and Takeuchi [11] term externalization—converting tacit practitioner knowledge into explicit artifacts that agents can consume. Our case study suggests that deliberate preparation can compress implementation time and reduce architectural misalignment, but we claim no generalizability from a single hackathon. What we do claim is that the alignment problem in agentic coding is fundamentally a preparation problem, and that the field would benefit from treating the work before agents write code as a first-class engineering discipline.

Context fluency offers a lens for understanding emerging practitioner skills—skills that bridge domain expertise, pedagogical design, and specification craft [2, 6]. We invite empirical validation, replication across domains, and the development of tools that support preparation-phase work as the field moves from ad-hoc agent interaction toward disciplined context engineering.

References

[1] Eman Abdullah Alomar et al. 2025. Promptware Engineering: The New Frontier of Software Engineering. *arXiv preprint* arXiv:2503.02400 (2025). doi:10.48550/arXiv.2503.02400

[2] Anthropic. 2025. Effective Context Engineering for AI Agents. Anthropic Documentation. https://docs.anthropic.com/en/docs/build-with-claude/context-engineering

[3] Arghavan Moradi Dakhel et al. 2025. A Taxonomy of Human-AI Collaboration in Software Engineering. *arXiv preprint* arXiv:2501.08774 (2025). doi:10.48550/arXiv.2501.08774

[4] Jeffrey Emanuel. 2025. beads_rust: High-Performance Task Tracking for AI Agents. GitHub repository. https://github.com/Dicklesworthstone/beads_rust

[5] Gartner. 2025. Context Engineering Is In, Prompt Engineering Is Out. Gartner Research Note. Gartner identified context engineering as a key shift in AI application development for 2025.

[6] GitHub. 2025. Spec Kit: Specification-Driven Development for AI Agents. GitHub repository. https://github.com/github/spec-kit

[7] Dex Horthy. 2025. The RPI Methodology: Research, Plan, Implement for Agentic Coding. HumanLayer Blog. https://humanlayer.dev/blog/rpi-methodology

[8] Geoffrey Huntley. 2025. The Ralph Loop: A Pattern for AI Agent Iteration. Blog post. https://ghuntley.com/ralph/

[9] Andrej Karpathy. 2025. Vibe Coding. Twitter/X post. https://x.com/karpathy/status/1886192184808149383 Term entered Merriam-Webster and Collins Dictionary in 2025.

[10] Hussein Mozannar et al. 2023. Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming. *arXiv preprint* arXiv:2210.14306 (2023). doi:10.48550/arXiv.2210.14306

[11] Ikujiro Nonaka and Hirotaka Takeuchi. 1995. *The Knowledge-Creating Company: How Japanese Companies Create the Dynamics of Innovation.* Oxford University Press, New York.

[12] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. 2023. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. *arXiv preprint* arXiv:2302.06590 (2023). doi:10.48550/arXiv.2302.06590

[13] Michael Polanyi. 1958. *Personal Knowledge: Towards a Post-Critical Philosophy.* University of Chicago Press, Chicago, IL.

[14] Michael Polanyi. 1966. *The Tacit Dimension.* University of Chicago Press, Chicago, IL.

[15] Marisa Vasconcelos, Jack Jamieson, Umang Bhatt, and Q. Vera Liao. 2024. Understanding Trust in AI-Assisted Code Generation. In *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAccT).* ACM, New York, NY.

[16] Veracode. 2025. State of Software Security: The Rise of AI Code. Industry report. Reports that 45% of AI-generated code contains security flaws, with a 10x spike in security findings from AI-generated code by June 2025.

[17] Grant Wiggins and Jay McTighe. 1998. *Understanding by Design.* Association for Supervision and Curriculum Development, Alexandria, VA.

[18] Steve Yegge. 2025. Beads: External Memory for AI Agents. GitHub repository. https://github.com/steveyegge/beads